

Block Circulant Codes for Ethereum PeerDAS

Birenjith Sasidharan^{1,2}, Emanuele Viterbo¹ and Dankrad Feist³

¹ Monash University

² Indian Institute of Technology Palakkad

³ Ethereum Foundation

biren@iitpkd.ac.in, emanuele.viterbo@monash.edu, dankrad.feist@ethereum.org

Abstract. In this report, we describe how block circulant (BC) codes can serve as an alternative to one-dimensional (1D) and two-dimensional (2D) Reed–Solomon (RS) codes in the Ethereum PeerDAS protocol. We begin by reviewing the implementation of 1D RS codes in PeerDAS and then present efficient encoding and reconstruction algorithms for BC codes, developed within the same implementation framework as the respective 1D RS algorithms. The proposed encoding algorithm also permits a graceful integration of KZG commitment scheme. Finally, we evaluate and compare the performance of BC codes in terms of code rate, stopping rate, and the size of the local codes they contain, against both 1D and 2D RS codes.¹

Keywords: Block Circulant Codes · DAS · Ethereum · PeerDAS

Contents

1	Introduction	2
2	Reed-Solomon Codes in PeerDAS	2
2.1	Overview of RS Codes	2
2.2	Stacked 1D RS Code	3
2.3	Reconstruction Algorithm	6
2.4	KZG Commitments for Stacked 1D RS Code	7
2.5	Important Code Parameters: Rate, Stopping rate and Local code size	7
2.6	2D RS Codes	8
3	Block Circulant Codes	8
3.1	A Description of the BC Code	8
3.2	Stacked BC code	9
4	Encoding and Reconstruction Algorithms for BC Codes	10
4.1	FFT over a group/coset restricted to a subgroup/subcoset	10
4.2	Encoding Algorithm	11
4.3	A Reconstruction Algorithm	13
4.4	KZG Commitments for Stacked BC Code	14
5	Performance Analysis	14

¹This work is supported by the Ethereum Foundation ESP Grant ID FY24-1745 titled “Improvements to 2D Reed-Solomon Codes in DAS”.

1 Introduction

The layer-2 (L2) scaling approach, particularly rollups, is Ethereum’s primary path toward widespread adoption, offering higher throughput and lower transaction costs without compromising security or decentralization. In rollups, transaction execution is moved off-chain, while the data necessary for verification is posted on-chain. To support this, EIP-4844 (Proto-Danksharding) introduced a temporary data storage mechanism called blobs (binary large objects) and a new transaction type known as the blob-carrying transaction. This upgrade provides a cost-efficient method for posting large volumes of rollup transaction data to Ethereum.

However, this development brings forth a crucial challenge: ensuring that the posted blobs are actually available across the network for verification for a certain duration of time. PeerDAS (EIP-7594) [WZ24] builds on EIP-4844 by implementing data availability sampling (DAS), enabling nodes to verify blob availability by downloading only a small fraction of the encoded data, leveraging the properties of erasure codes. This significantly reduces per-node load, thereby increasing Ethereum’s data capacity for rollups. Random sampling of erasure-coded data combined with vector commitment schemes, as formalized in [ASBK21], forms the core of the PeerDAS protocol, which was included in Ethereum’s Fusaka upgrade activated on December 3, 2025. A combination of one-dimensional (1D) Reed–Solomon (RS) erasure code and Kate–Zaverucha–Goldberg (KZG) vector commitment has emerged as the primary construction within this protocol.

In this report, we identify the design parameters that are pivotal to engineering an erasure code suitable for PeerDAS. With the goal of optimizing these parameters, we propose an optimized variant of block circulant (BC) codes [SVHD25] as a replacement candidate for 1D RS codes. We develop efficient encoding and reconstruction algorithms and present techniques to efficiently integrate BC codes with KZG algorithms.

We begin with a review of how 1D RS codes are implemented in the current PeerDAS protocol in Sec. 2. Sec. 3 provides a brief introduction to BC codes. Encoding and reconstruction algorithms for the BC code are presented in Sec. 4. Finally, Sec. 5 compares the performance of BC codes against 1D and 2D RS codes with respect to the key parameters relevant to PeerDAS.

2 Reed-Solomon Codes in PeerDAS

2.1 Overview of RS Codes

Let $n, k \leq n$ be positive integers. Let $\mathbb{F}_q$ be a finite field of $q > n$ elements. Let $\{\alpha_0, \alpha_1, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_q$ be an ordered set and $(\beta_0, \beta_1, \dots, \beta_{n-1}) \subseteq (\mathbb{F}_q \setminus \{0\})^n$. The first is called a set of evaluation points and the second is called a vector of non-zero multipliers. Then we define generalized Reed-Solomon code as:

$$\mathcal{C}_{\text{GRS}} = \{(\beta_0 f(\alpha_0), \beta_1 f(\alpha_1), \dots, \beta_{n-1} f(\alpha_{n-1})) \mid f(X) \in \mathbb{F}_q[X], \deg(f(X)) \leq k-1\}. \quad (1)$$

If $\beta_i = 1$ for all i , then we call $\mathcal{C}_{\text{GRS}}$ a Reed-Solomon code and denote it as $\mathcal{C}_{\text{RS}}$. Every element $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}_{\text{GRS}}$ is called a codeword and each c_i is called a coded symbol of $\mathbf{c}$. Since $k-1 < n$ and every polynomial of degree $(k-1)$ is uniquely determined by k distinct evaluations, every codeword is uniquely associated with a polynomial $f(X)$ of degree $k-1$.

For any arbitrary $\mathbf{a} = (a_0, a_1, \dots, a_{k-1}) \in \mathbb{F}_q^k$, we encode it into a codeword as follows:

$$(c_0, c_1, \dots, c_{n-1}) = (a_0, a_1, \dots, a_{k-1}) \begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha_0 & \alpha_1 & \dots & \alpha_{n-1} \\ \vdots & \vdots & & \vdots \\ \alpha_0^{r-1} & \alpha_1^{r-1} & \dots & \alpha_{n-1}^{r-1} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \ddots \\ \beta_{n-1} \end{bmatrix}. \quad (2)$$

It may be noted that $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$ is the codeword associated to the polynomial $f(X) = a_0 + a_1X + \dots + a_{k-1}X^{k-1}$.

All coded symbols do obey certain linear constraints that can be expressed with the help of polynomials of degree at most $(n - k - 1)$. Specifically, it can be shown that for every $p(X) \in \mathbb{F}_q[X]$ such that $\deg(p(X)) \leq n - k - 1$,

$$c_0\beta'_0p(\alpha_0) + c_1\beta'_1p(\alpha_1) + \dots + c_{n-1}\beta'_{n-1}p(\alpha_{n-1}) = 0, \quad (3)$$

where

$$(\beta'_i)^{-1} = \beta_i \prod_{\substack{j=0 \\ j \neq i}}^{n-1} (\alpha_i - \alpha_j), \quad i = 0, 1, \dots, n-1. \quad (4)$$

Furthermore in (3), it is possible to identify $(n - k - 1)$ different polynomials of degree at most $(n - k - 1)$ with suitable evaluations at $\{\alpha_0, \alpha_1, \dots, \alpha_{n-1}\}$ so that the codeword $\mathbf{c}$ satisfies the following full-rank system of linear (parity-check) equations.

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ \alpha_0 & \alpha_1 & \dots & \alpha_{n-1} \\ \vdots & & \ddots & \vdots \\ \alpha_0^{n-k-1} & \alpha_1^{n-k-1} & \dots & \alpha_{n-1}^{n-k-1} \end{bmatrix} \begin{bmatrix} \beta'_0 & & & \\ & \beta'_1 & & \\ & & \ddots & \\ \vdots & & & \beta'_{n-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{bmatrix} = \mathbf{0} \quad (5)$$

2.2 Stacked 1D RS Code

In Ethereum PeerDAS, the Reed–Solomon codes are carefully optimized for data encoding; we refer to this optimized construction as the stacked 1D RS code. We shall first describe how data is stored as blobs (abbreviation for binary large objects), and subsequently explain the encoding and reconstruction algorithms.

2.2.1 Data in Blobs

Rollup sequencer nodes post data containing transactions to the Ethereum Layer-1 (L1) network as blobs. They also post to the L1 network a special blob-carrying transaction containing a cryptographic commitment to the blob. The blob-carrying transaction remains permanently on the Ethereum main chain, while the blobs are stored only for about 18 days and subsequently removed. The way blobs propagate through, and are stored in the L1 network, is different from how blocks are handled. The Ethereum full nodes run a DAS protocol to ensure that all the blob data is indeed available in order to ensure the validity of the blob-carrying transaction. In order to run the DAS protocol, the blob data are first erasure-coded using RS codes.

Consider the illustrative example in Figure 1. In this example, every blob (e.g., Blob 0 as shown in the figure) has $k = 2$ cells. Each cell is a vector of $D = 2$ symbols from a finite field $\mathbb{F}_q$, e.g., $\text{Cell}(0, 0) \in \mathbb{F}_q^D$. Therefore every blob is an element of $\mathbb{F}_q^{D \times k}$. Every blob is encoded to obtain an extended blob, which will be explained in detail in the next subsection. The extended blob is an element of $\mathbb{F}_q^{D \times n}$ and in this example $n = 4$. Multiple extended blobs are arranged as an array containing $M = 4$ rows and therefore each column is an element of $\mathbb{F}_q^{MD}$. Ethereum full nodes store columns of the array, with various custody requirements for various nodes. A non-validating full node must have custody of 4 columns, whereas a validator full node must have 8 columns. A super node will have a custody of all the columns. In the DAS protocol, every query demands a particular column, i.e., a vector in $\mathbb{F}_q^{MD}$, and a node that has the custody of the particular column may respond to that query by transmitting that column. Since the query is made as part of random sampling, a column is sometimes referred to as a sample.

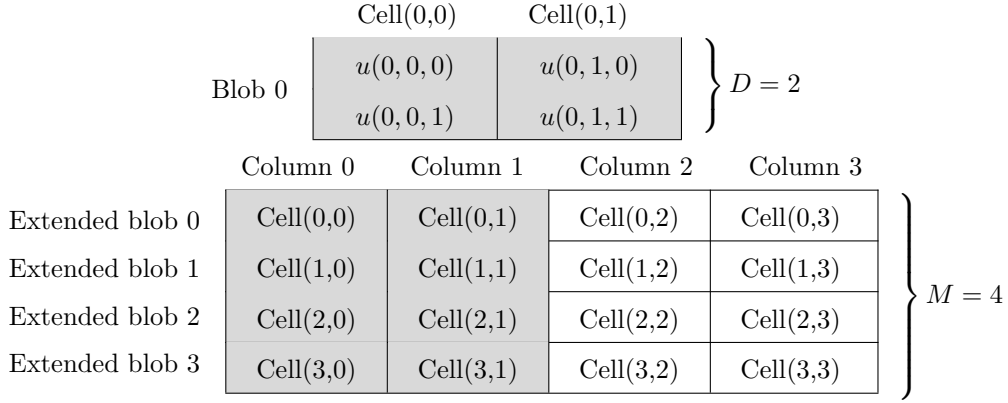


Figure 1: The structure of blobs, extended (encoded) blobs and columns. Each $u(\cdot, \cdot, \cdot)$ is an element of a finite field $\mathbb{F}_q$. In this example, $k = 2, D = 2, n = 4, M = 4$.

2.2.2 Efficient Encoding of Blobs

We use the same example parameters in Fig. 1 to describe the encoding using stacked 1D RS code. For every blob, we have $k = 2, D = 2$, and thus we have $kD = 4$ finite field symbols. In the example, we have $n = 4$ and we first construct a $[nD = 8, kD = 4]$ RS code over a finite field $\mathbb{F}_q$ as follows. First, we choose a prime field $\mathbb{F}_q$, i.e., q is prime, such that $2^t \mid (q - 1)$ and $2nD \mid 2^t$ for some t^2 . In this example, $\mathbb{F}_{17}$ meets the requirements with $t = 4$.

Consider the subgroup $\mathbb{H}_0 \subseteq \mathbb{F}_q^*$ of 2^t elements and let ω be its generator. Clearly, $\mathbb{F}_q^*$ is cyclic and so is $\mathbb{H}_0$. In this example, we have $\mathbb{H}_0 = \mathbb{F}_{17}^*$ and ω is any primitive element of $\mathbb{F}_{17}$. We first enlist subgroups of $\mathbb{H}_0$ such that the smallest subgroup $\mathbb{H}_3$ is of size $D = 2$:

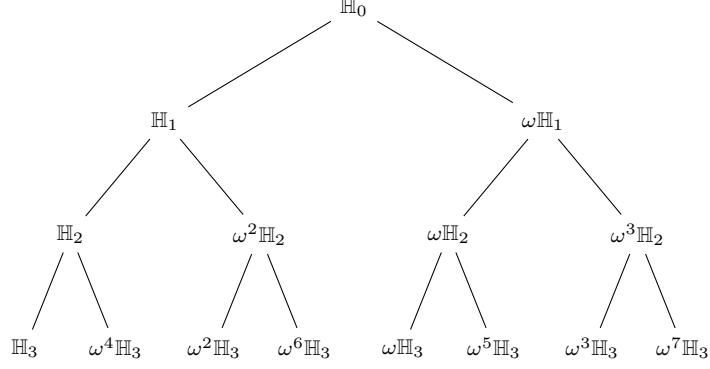
$$\begin{aligned}
 \mathbb{H}_0 &= \mathbb{F}_{17}^* = \{1, \omega, \omega^2, \dots, \omega^{15}\} \\
 \mathbb{H}_1 &= \{1, \omega^2, \omega^4, \dots, \omega^{14}\} \\
 \mathbb{H}_2 &= \{1, \omega^4, \omega^8, \omega^{12}\} \\
 \mathbb{H}_3 &= \{1, \omega^8\}.
 \end{aligned}$$

A tree is constructed identifying nodes as subgroups of $\mathbb{H}_0$ and their cosets, while edges identify a containment relation between these sets. We refer to it as the subgroup tree and it is given in Fig. 2 for the example being discussed.

Let $f(X) \in \mathbb{F}_{17}[X]$ with $\deg(f) < kD = 4$. An ordered set of nD evaluation points are chosen in such a manner that (a) the set of evaluation points corresponding to the first cell $\text{Cell}(0, 0)$ is a subgroup $\mathbb{H}$ of $\mathbb{H}_0$ and $|\mathbb{H}| = D$; (b) each cell from $\text{Cell}(0, 1)$ to $\text{Cell}(0, n - 1)$ are respectively associated to cosets of $\mathbb{H}$ picked by traversing from the left to the right in the subgroup tree. The set of D code symbols associated to each cell are obtained by evaluating f at corresponding subgroup or the coset as the case may be. This results in a codeword of the stacked 1D RS code. In the example, the subgroup/coset associated to each cell are listed below:

$$\text{Cell}(0, 0) \Leftrightarrow \mathbb{H}_3, \text{Cell}(0, 1) \Leftrightarrow \omega^4 \mathbb{H}_3, \text{Cell}(0, 2) \Leftrightarrow \omega^2 \mathbb{H}_3, \text{Cell}(0, 3) \Leftrightarrow \omega^6 \mathbb{H}_3.$$

²It is sufficient that $nD \mid 2^t$ for the code construction, however, we will see later that $2nD \mid 2^t$ is required to develop an efficient reconstruction algorithm. See Step 6 in the algorithm given in Sec. 2.3.

**Figure 2:** Tree of subgroups and cosets of $\mathbb{H}_0 = \mathbb{F}_{17}^*$.

$\mathbb{H}_3$	$\omega^4 \mathbb{H}_3$	$\omega^2 \mathbb{H}_3$	$\omega^6 \mathbb{H}_3$
$f(1)$	$f(\omega^4)$	$f(\omega^2)$	$f(\omega^6)$
$f(\omega^8)$	$f(\omega^{12})$	$f(\omega^{10})$	$f(\omega^{14})$

Figure 3: Codeword (extended blob 0) obtained by encoding blob 0 by $([n = 4, k = 2], D = 2)$ 1D stacked RS code.

Hence we obtain the codeword (extended blob) as given in Fig. 3. In exactly similar lines each of the M blobs is encoded and arranged as in Fig. 1. The code described above is referred to as the 1D stacked RS code with parameters $([n = 4, k = 2], D = 2)$ repeated for $M = 4$ rows.

In the following, we present a low-complexity algorithm used in PeerDAS to evaluate the polynomial f at the selected points in $\mathbb{H}_0$. Let $m = 2^t$. Then the annihilating polynomial of $\mathbb{H}_0$ is

$$z(X) = X^m - 1.$$

The Lagrange basis of polynomials corresponding to $\mathbb{H}_0$ can be computed in a straightforward manner as:

$$L_i(X) = \frac{\omega^i}{m} \frac{X^m - 1}{X - \omega^i}, \quad i \in [m].$$

Let

$$r = 2^{t-j}, \quad \beta = \omega^{2^j}, \quad j = 0, 1, \dots, t-1.$$

The subgroup $\mathbb{H}_j = \{\beta^i, i \in [r]\}$ for $j = 0, 1, \dots, t-1$. The annihilating polynomial of a coset $\omega^s \mathbb{H}_j$, $0 \leq s \leq 2^j - 1$ is given by

$$z_{js}(X) = X^r - (\omega^s)^r.$$

The Lagrange basis of polynomials corresponding to the coset $\omega^s \mathbb{H}_j$ is given by

$$L_i^{js}(X) = \frac{\beta^i}{r \omega^{s(r-1)}} \cdot \frac{X^r - (\omega^s)^r}{X - \omega^s \beta^i}, \quad i = 1, 2, \dots, r.$$

Consider an arbitrary polynomial $g(X) \in \mathbb{F}_q[X]$ of degree $d < r$. Suppose that we know r evaluations of $g(X)$ at the coset $\omega^s \mathbb{H}_j$. Let these evaluations be given as:

$$g(\omega^s \beta^i) = \hat{a}_{i-1}, \quad i \in [r].$$

Then we can interpolate $g(X)$ as

$$g(X) = \sum_{i \in [r]} \hat{a}_{i-1} L_i^{js}(X).$$

Let $g(X) = a_0 + a_1 X + \dots + a_{r-1} X^{r-1}$, $\mathbf{a} = [a_0, a_1, \dots, a_{r-1}]^t$ and $\hat{\mathbf{a}} = [\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{r-1}]^t$. Then,

$$\begin{aligned} \hat{\mathbf{a}} &= \begin{bmatrix} 1 & \beta & \beta^2 & \dots & \beta^{(r-1)} \\ 1 & \beta^2 & \beta^4 & \dots & \beta^{2(r-1)} \\ 1 & \beta^3 & \beta^6 & \dots & \beta^{3(r-1)} \\ \vdots & \vdots & & \ddots & \vdots \\ 1 & \beta^{(r-1)} & \beta^{2(r-1)} & \dots & \beta^{(r-1)^2} \end{bmatrix} \begin{bmatrix} 1 & & & & \\ & \omega^s & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \omega^{(r-1)s} \end{bmatrix} \mathbf{a} \\ &\triangleq F(\beta, r) \cdot A \cdot \mathbf{a}. \end{aligned}$$

Since β is r -th root of unity, the matrix $F(\beta, r)$ is a matrix of discrete Fourier transform. Transformation by $F(\beta, r)$ or its inverse can be implemented by fast Fourier transform (FFT) algorithm with $O(r \log(r))$ complexity. Multiplication by A or A^{-1} takes $O(r)$ operations and thus the complexity remains as $O(r \log(r))$. Therefore, both the evaluation map taking $\mathbf{a}$ to $\hat{\mathbf{a}}$ and the interpolation map taking $\hat{\mathbf{a}}$ to $\mathbf{a}$ can be implemented using FFT algorithm with $O(r \log(r))$ field operations. In the example discussed above, we need to compute one FFT over the group $\mathbb{H}_1$ with $r = |\mathbb{H}_1| = 8$ in order to encode one blob. Thus we need a total of M FFTs are needed.

2.3 Reconstruction Algorithm

In the example, the group $\mathbb{H}_1$ of size nD consists of all evaluation points of the $([n, k], D)$ stacked 1D RS code. Let $J \subset \mathbb{H}_1, |J| = (n - k)D$ be the set of points whose evaluations are erased. Let $f(X)$ be the polynomial obtained by interpolating available kD evaluations over $\mathbb{H}_1$. Our interest is to recover $f(X)$. An efficient reconstruction algorithm to reconstruct these evaluations is presented in [But18] and is used in PeerDAS. Suppose $\hat{\mathbf{r}} = (\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2, \dots, \hat{\mathbf{r}}_n) \in (\mathbb{F}_q^D)^n$ be the extended blob subject to erasures. In DAS, we will have missing columns restricting the possible patterns of erasures. The algorithm below (as presented in [WZ24]) assumes the above type of erasure patterns and reconstructs the polynomial $f(X)$ efficiently.

1. Compute $Z(X) = \prod_{j \in J} (X - j)$ with $\deg(Z) \leq |J|$, the vanishing polynomial of all elements in $\mathbb{H}_1$ associated to the erasure locations.
2. Define a vector $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n) \in (\mathbb{F}_q^D)^n$ such that $\mathbf{e}_j = 0$ if $j \in J$ or $\mathbf{e}_j = \mathbf{r}_j$ if $j \notin J$.
3. Compute the polynomial $E(X)$ with $\deg(E) \leq nD - 1$, which interpolates vector $\mathbf{e}$ over $\mathbb{H}_1$.
4. Compute the evaluations of $E(X)Z(X)$ over $\mathbb{H}_1$. This can be done efficiently because evaluations of $E(X)$ are readily available, and of $Z(X)$ can easily computed from its definition in Step 1.

5. Compute $f(X)Z(X)$ in coefficient form. To do so, note that $\deg(fZ) \leq nD - 1$, and so we can recover $f(X)Z(X)$ by interpolating evaluations of $E(X)Z(X)$ over $\mathbb{H}_1$. (Observe that $f(x)Z(x) = E(x)Z(x)$ for all $x \in \mathbb{H}_1$.)
6. Divide $f(X)Z(X)$ by $Z(X)$. This can be done inefficiently in the following way:
 - (a) Evaluate $f(X)Z(X)$ over $\beta\mathbb{H}_1$.
 - (b) Evaluate $Z(X)$ over $\beta\mathbb{H}_1$. This can be easily done given the form of $Z(X)$.
 - (c) Obtain evaluations of $f(X)$ over $\beta\mathbb{H}_1$ by dividing $f(x)Z(x)$ by $Z(x)$ for every $x \in \beta\mathbb{H}_1$. (Observe that $Z(x) \neq 0$ for every $x \in \beta\mathbb{H}_1$.)
 - (d) Interpolate $f(X)$ from its evaluations over $\beta\mathbb{H}_1$.

In the above, all evaluations (except those of $Z(X)$ which can be easily computed with linear complexity.) and interpolations can be done efficiently using FFTs and IFFTs.

2.4 KZG Commitments for Stacked 1D RS Code

The polynomial commitment scheme introduced by Kate, Zaverucha and Goldberg [KZG10] allows to commit to a polynomial $f \in \mathbb{F}_q[X]$ of bounded degree d . Both the commitment and the proof for an evaluation of f are of constant size and therefore the scheme is succinct. In PeerDAS, it is required to obtain a proof for all the evaluations within a cell simultaneously and hence we call it a multiproof. A formal description of how to construct such a KZG multiproof and its security analysis for the stacked 1D RS code is available in [WZ24]. The ability to generate a constant-sized multiproof solely relies on the fact that each cell is formed by evaluations of the message polynomial on either a subgroup or a coset of a subgroup. As established formally in [WZ24], the security is not compromised even if the DAS protocol involves querying a column consisting of M cells.

2.5 Important Code Parameters: Rate, Stopping rate and Local code size

We are interested in optimizing with respect to three parameters while choosing an erasure code for the DAS protocol.

1. Rate (R): The rate of the code $R = (kD/nD) = k/n$. We would like to increase the rate so that the storage overhead in extending every blob is reduced.
2. Stopping rate (S): The RS code can recover from any set of $(nD - kD)$ erasures. In particular, it can recover any missing $(n - k)$ columns. The stopping rate is defined as the fraction of tolerable erasures, i.e., $S = \frac{nD - kD}{nD} = 1 - R$, exhibiting a tradeoff between R and S . Information theoretically, this is best possible tradeoff achievable for any erasure code.
3. Local code size (L): If a proper subset of coded symbols of an $[n, k]$ linear code forms another $[n_0 < n, k_0 < k]$ code, then we call it a local code. In a stacked 1D RS code with parameters $([n, k], D)$, there is no linear constraint that binds together less than $kD + 1$ symbols. Every redundant symbol in an RS codeword (extended blob) is modified if at least one of the data symbol in the blob is modified. It follows that an RS code does not have any local code of smaller length. For a code containing local codes, it may be possible to reconstruct a missing symbol by accessing a smaller set of symbols. Therefore, nodes with a custody of only a few columns (i.e., which are not super nodes) can also take part in the reconstruction process making it more decentralized. Furthermore, the computational complexity of KZG commitments and multiproofs increases with blocklength. By verifying a sampled column by means

of a KZG commitment/multiproof with respect to a local code, the complexity of verification can be reduced. For the above reasons, it is desirable to have local codes of smaller lengths. The absence of any local codes therefore becomes a drawback of the stacked $([n, k], D)$ 1D RS code.

2.6 2D RS Codes

In [ASBK21], 2D RS codes consisting of many shorter local codes have been introduced as a replacement to 1D RS code. These local codes are 1D RS codes so that KZG commitment scheme can be employed for each of the local code. The construction of 2D RS code builds on top of an 1D RS code. Consider Fig. 3 in which M 1D RS codewords are arranged in rows. The 2D RS code is obtained by extending each of the column with redundant cells, so that every row/column becomes an 1D RS code. In this construction, every row/column becomes a local code of a bigger global code. We use $([n_0, k_0], D)$ to denote parameters of each row/column local code. Then the $([n, k], D)$ 2D RS code can be shown to have:

$$\begin{aligned} R &= (k_0 D)^2 / (n_0 D)^2 = (k_0 / n_0)^2 \\ S &= ((n_0 D - k_0 D + 1)^2 - 1) / (n_0 D)^2 \approx (1 - \sqrt{R})^2 \\ L &= n_0 D \end{aligned}$$

Clearly, the 2D RS code has $2n_0$ local codes, each of size $L = n_0 D \ll nD$, which is a clear advantage over 1D RS code. However, the tradeoff between S and R becomes worse compared 1D RS code. The decay of stopping rate as a function of rate is significantly faster than 1D RS code. Therefore, the storage overhead required in a 2D RS code is much larger than the 1D RS code to attain the same stopping rate.

3 Block Circulant Codes

Block circulant (BC) codes are introduced in [SVHD25] as a viable alternative to both 1D (currently used) and 2D RS codes (expected to be incorporated later) in PeerDAS. Similar to the 2D RS code, the BC code contains multiple local codes, and each local code can be designed as a $([n_0, k_0], D)$ stacked 1D RS code. Therefore, it is compatible with the KZG scheme. It turns out that BC codes achieve certain operating regimes of rate, stopping rate and local code size that are not feasible with either a 1D or a 2D RS code.

In a 2D RS code, the symbols are arranged in a two-dimensional square grid and there are linear constraints binding symbols belonging to every row or column. In the case of a BC code, symbols are arranged on a circle and the circle is divided into multiple overlapping arcs. Every arc has two neighboring overlapping arcs, one on the left and the other on the right assuming a clockwise direction. Symbols belong to every arc are subject to a set of linear constraints. We present a more concrete description in the next subsection.

3.1 A Description of the BC Code

Consider the example consisting of $n = 16$ symbols as given in Fig. 4. There are $\mu = 4$ arcs (two of them marked by closed curves) each containing $n_0 = 6$ symbols, equally spaced on the circle. There are a total of $k = 8$ unconstrained symbols denoted as $d_1, \dots, d_8$. Each local code contains $k_0 = 4$ unconstrained symbols. The remaining symbols are generated subject to certain linear constraints. In every arc, $\rho = n_0 - k_0 = 2$ redundant symbols are added in such a manner that the local code corresponding to the arc becomes a 1D RS code. For example, the vector of symbols $(d_1, d_2, p_{11}, p_{12}, d_3, d_4)$ corresponding to a single arc forms a codeword of an 1D RS code. Thus an arc is uniquely identified with

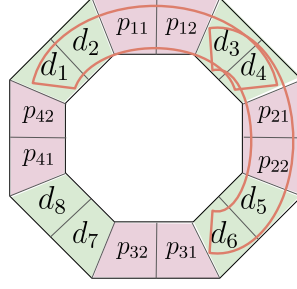


Figure 4: Illustration of the block circulant code. In this example, information symbols in the green background color are combined with parity symbols in the pink background color region resulting in the final arrangement. The groupings of symbols within the closed curves indicate local codes.

a local code, and we may write arc and local code interchangeably. There lies exactly $\omega = 2$ symbols at the overlapping region of two adjacent local codes and therefore we call ω as the overlap width. Every unconstrained symbol is part of two distinct local codes and we call it the overlapping factor, denoted by λ . In the present arrangement, we have $\lambda = 2$. In the general BC code presented in [SVHD25], arrangements with higher values of λ are possible. Other parameters like ω and ρ can also be modified suitably. It is shown in [SVHD25] that a BC code with $\lambda = 2, 3$, the total number of erasures that can recovered by the BC code has been shown to be $\lambda\rho$. The ratio of total number of tolerable erasures to n is called the stopping rate and is denoted by S . We summarize the notations used for adjustable parameters of the BC code in the table below.

Parameters of BC code	
Notation	Meaning of the parameter(s)
μ	Number of local codes
λ	Overlap factor (Every symbol is part of λ local codes)
ρ	Number of redundant symbols in a local code
ω	Overlap width (Number of symbols two adjacent local codes overlap on)
$[n_0, k_0] = [\lambda\omega + \rho, \lambda\omega]$	Local code blocklength and dimension
$[n, k] = [\mu(\omega + \rho), \mu\omega]$	Global code blocklength and dimension
$R = (k/n) = \frac{\omega}{\omega + \rho}$	Rate
$S = \lambda\rho/\mu(\omega + \rho)$	Stopping rate
$R_0 = (k_0/n_0) = \frac{\lambda\omega}{\lambda\omega + \rho}$	Local code rate
$S_0 = \rho/(\lambda\omega + \rho)$	Local code stopping rate

3.2 Stacked BC code

In Sec. 2.2.2, we described how an 1D RS code is optimized to construct a stacked 1D RS code. The exact procedure is applicable to the BC code as well because (a) each of the local code in the BC code is an 1D RS code, (b) the encoding can be performed by a series of local 1D RS encoders with carefully chosen set of evaluation points for each of the local code. In the next section, we will describe the encoding procedure for a *stacked BC code* with the help of an example.

4 Encoding and Reconstruction Algorithms for BC Codes

In this section, we present FFT-based low-complexity algorithms for encoding and reconstruction in stacked BC code. Both the algorithms will assume that $\lambda = 2$. Before we describe the algorithms, we present an efficient method to compute a subset of FFT values without computing the entire FFT vector. This method will later become useful in arriving at an efficient BC encoder.

4.1 FFT over a group/coset restricted to a subgroup/subcoset

We have described FFT over a group/coset in Sec 2.2.2. In order to implement BC encoding, we will require to compute values of an FFT vector over group/coset only at a proper subset of locations. If the subset turns out to be subgroup (subcoset), then we refer to as FFT over a group (coset) restricted to a subgroup (subcoset). This task can be achieved with less number of operations than required to compute the entire FFT vector.

Consider FFT of $\mathbf{a}$ over a coset $\omega^s \mathbb{H}_j$ given by

$$\hat{\mathbf{a}} = \begin{bmatrix} 1 & \beta & \beta^2 & \dots & \beta^{(r-1)} \\ 1 & \beta^2 & \beta^4 & \dots & \beta^{2(r-1)} \\ 1 & \beta^3 & \beta^6 & \dots & \beta^{3(r-1)} \\ \vdots & \vdots & & \ddots & \vdots \\ 1 & \beta^{(r-1)} & \beta^{2(r-1)} & \dots & \beta^{(r-1)^2} \end{bmatrix} \begin{bmatrix} 1 & & & & \\ & \omega^s & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \omega^{(r-1)s} \end{bmatrix} \mathbf{a},$$

where $\beta = \omega^{2^j}$ and $r = 2^{t-j}$ for $j = 0, 1, \dots, t-1$ and $0 \leq s \leq 2^j - 1$. Suppose that we are required to compute $\hat{\mathbf{a}}$ at points restricted to $\omega^u \mathbb{H}_{j+1} \subset \omega^s \mathbb{H}_j$ where $u \in \{s, s + 2^j\}$. We call it FFT of $\mathbf{a}$ over $\omega^s \mathbb{H}_j$ restricted to $\omega^u \mathbb{H}_{j+1}$ and denote it as $(\hat{\mathbf{a}}(\alpha), \alpha \in \omega^u \mathbb{H}_{j+1})$. In the following, we will see how to compute these $|\omega^u \mathbb{H}_{j+1}|$ values of $\hat{\mathbf{a}}$ without computing the entire $\hat{\mathbf{a}}$.

If $u = s$, then consider $i = 2m$ for $m = \{0, 1, \dots, (r/2) - 1\}$. Keeping in mind $\beta^r = 1$, we can write

$$\begin{aligned} \hat{a}_{2m} &= a_0 + a_1 \omega^s \beta^{2m} + a_2 \omega^{2s} \beta^{4m} + \dots + a_{r-1} \omega^{s(r-1)} \beta^{(r-1)2m} \\ &= a_0 + a_1 \omega^s \beta^{2m} + \dots + a_{(r/2)-1} \omega^{s((r/2)-1)} \beta^{((r/2)-1)2m} + \\ &\quad a_{(r/2)} \omega^{sr/2} \beta^{(r/2)2m} + a_{(r/2)+1} \omega^{s((r/2)+1)} \beta^{((r/2)+1)2m} + \dots + a_{r-1} \omega^{s(r-1)} \beta^{(r-1)2m} \\ &= \sum_{\ell=0}^{(r/2)-1} a_\ell \omega^{s\ell} (\beta^2)^{m\ell} + \sum_{\ell=0}^{(r/2)-1} a_{(r/2)+\ell} \omega^{s((r/2)+\ell)} (\beta^2)^{m\ell} \\ &= \sum_{\ell=0}^{(r/2)-1} [a_\ell + \omega^{sr/2} a_{(r/2)+\ell}] \omega^{s\ell} (\beta^2)^{m\ell} \end{aligned} \tag{6}$$

If $u = s + 2^j$, then consider $i = 2m + 1$ for $m = \{0, 1, \dots, (r/2) - 1\}$. In similar lines, we obtain:

$$\begin{aligned} \hat{a}_{2m+1} &= a_0 + a_1 \omega^s \beta^{2m+1} + a_2 \omega^{2s} \beta^{4m+2} + \dots + a_{r-1} \omega^{s(r-1)} \beta^{(r-1)(2m+1)} \\ &= a_0 + \beta \omega^s a_1 \beta^{2m} + (\beta \omega^s)^2 a_2 \beta^{4m} + \dots + (\beta \omega^s)^{r-1} a_{r-1} \beta^{(r-1)2m} \\ &= \sum_{\ell=0}^{(r/2)-1} [a_\ell + \omega^{sr/2} a_{(r/2)+\ell}] (\beta \omega^s)^\ell (\beta^2)^{m\ell} \end{aligned} \tag{7}$$

We can compute (6) by invoking FFT on a $(r/2)$ -length vector $[a_\ell + \omega^{sr/2} a_{(r/2)+\ell}]$, $\ell = 0, 1, \dots, (r/2) - 1$ over $\omega^s \mathbb{H}_{j+1}$. Similarly, we can compute (7) by invoking FFT on a $(r/2)$ -length vector $[a_\ell + \omega^{sr/2} a_{(r/2)+\ell}]$, $\ell = 0, 1, \dots, (r/2) - 1$ over $(\beta \omega^s) \mathbb{H}_{j+1}$. It follows that

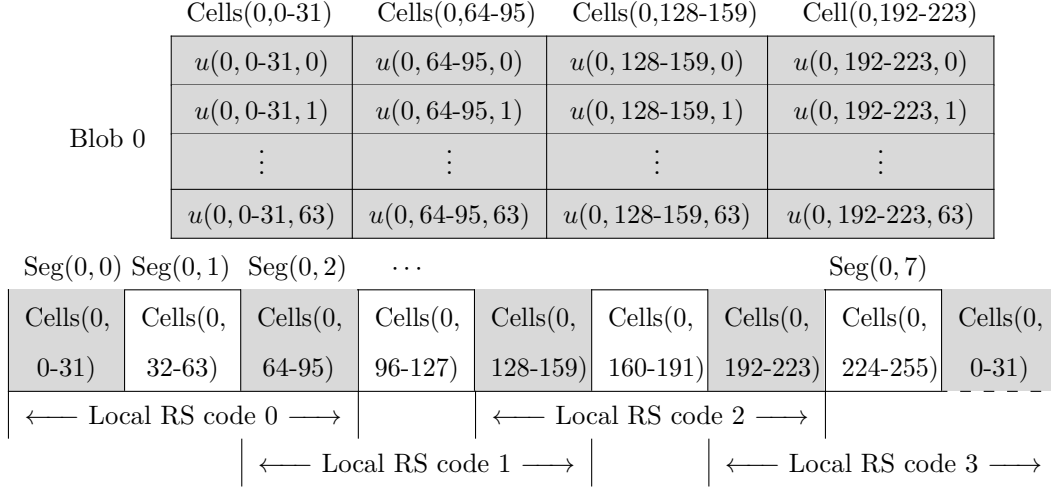


Figure 5: Structure of extended (encoded) blob 0 in BC code.

computation of the restricted FFT can be performed in $O((r/2)(\log(r/2) + 1))$ operations, much less than $O(r \log(r))$ operations that are needed to compute the entire FFT vector.

4.2 Encoding Algorithm

We describe the encoding with the help of an example illustrated in Fig. 5. Here, overlap factor $\lambda = 2$ and there are $\mu = 4$ local codes. We pick the stacking parameter $D = 64$ and a blob consists of 8192 finite field symbols arranged in $k = 128$ cells. Each blob is an element of $\mathbb{F}_q^{(D \times k)} = \mathbb{F}_q^{(64 \times 128)}$, whereas each cell belongs to $\mathbb{F}_q^{64}$. First we partition 128 cells of Blob 0 into $\mu = 4$ segments each containing $\omega = 32$ cells. These four segments are indexed as Segment(0, 0), Segment(0, 2), Segment(0, 4), and Segment(0, 6). During the encoding procedure, we shall generate and place segments containing redundant symbols in between two successive (Segment(0, 6) is considered to be adjacent to Segment(0, 0), viewing them cyclically) segments. The resultant 8 segments indexed as Segment(0, 0) to Segment(0, 7) jointly form the extended (encoded) blob. In general, the redundant segments will have ρ cells, but in this example, we have $\rho = \omega = 32$ since the rate is chosen to be $R = 0.5$. Therefore, every segment of the extended blob consists of 32 cells.

In Fig. 5, the blob 0 with 128 cells are shown. Anticipating the structure of extended blob, the second index of cells associated to Blob 0 are kept as 0-31, 64-95, 128-159, 192-223. Cells within extended blob 0 with the above indices are retained with the same data as blob 0. The cells indexed as 32-63, 96-127, 160-191, 224-255 in the extended blob 0 will be populated with redundant data during encoding.

We choose a prime field $\mathbb{F}_q$ such that $\mathbb{H}_3 \subset \mathbb{H}_2 \subset \mathbb{H}_1 \subset \mathbb{H}_0 \subset \mathbb{F}_q^*$ is a chain of subgroups in $\mathbb{F}_q^*$ satisfying size constraints

$$|\mathbb{H}_3| = D = 64, |\mathbb{H}_2| = \omega D = 2048, |\mathbb{H}_1| = 2\omega D = 4096, |\mathbb{H}_0| = 2\lambda\omega D = 8192.$$

It may be observed that the prime number q associated to the elliptic curve **bls12-381** satisfies that $2^{32} \mid (q - 1)$ and is therefore suitable to find the above subgroup chain. The tree of subgroups is given in Fig. 6. It matches with Fig. 2 except that sizes of each subgroup are different and the last level has more nodes (4×64 leaves). Next, we describe how the extended blob 0 is constructed.

1. The code consists of 4 local codes indexed as 0-3, and encoding is carried out by

separately encoding each of the local code one by one. In Fig. 5, cells forming each of the local codes are clearly indicated.

2. The first $3\omega = 96$ cells Cells(0, 0-95) comprising of $3\omega D = 96 \times 64 = 6144$ finite field symbols form a $([n_0 = 96, k_0 = 64], D = 64)$ stacked 1D RS codeword. A polynomial $f_0(X) \in \mathbb{F}_q[X]$ of degree at most $2\omega D = 4096$ may be formed using symbols in blob 0 data at Cells(0, 0-31) and Cells(0, 64-95). Each cell in the extended blob is associated to a coset of $\mathbb{H}_3$ as follows:

$$\begin{aligned} \text{Cells}(0, 0-31) &\Leftrightarrow \mathbb{H}_3, \beta^{64}\mathbb{H}_3, \beta^{32}\mathbb{H}_3, \dots, \beta^{124}\mathbb{H}_3 \\ \text{Cells}(0, 64-95) &\Leftrightarrow \beta^2\mathbb{H}_3, \beta^{66}\mathbb{H}_3, \beta^{34}\mathbb{H}_3, \dots, \beta^{126}\mathbb{H}_3 \\ \text{Cells}(0, 32-63) &\Leftrightarrow \beta\mathbb{H}_3, \beta^{65}\mathbb{H}_3, \beta^{33}\mathbb{H}_3, \dots, \beta^{125}\mathbb{H}_3 \end{aligned}$$

In each of 96 cells, $f_0(X)$ is evaluated at the corresponding coset and the evaluations become the contents of that cell. This yields a stacked RS codeword of the Local code 0. The polynomial $f_0(X)$ is constructed in such a manner that the evaluations at Cells(0, 0-31) and Cells(0, 32-63) in the extended blob are exactly same as the corresponding values in the blob. This is achieved by taking the data values as evaluations and interpolating a polynomial from these data values.

3. The same procedure is repeated for Local code 1, 2 and 3. The corresponding polynomials are denoted by $f_1(X)$, $f_2(X)$ and $f_3(X)$. The blob data used to construct each $f_i(X)$, $i = 1, 2, 3$, cells associated to each local code, and the cosets used as evaluation points are listed below.

(a) Local code 1: Cells(0, 64-159)

$$\begin{aligned} f_1(X) &\Leftrightarrow \text{Cells}(0, 64-95) \text{ and Cells}(0, 128-159) \text{ of blob 0} \\ \text{Cells}(0, 64-95) &\Leftrightarrow \beta^2\mathbb{H}_3, \beta^{66}\mathbb{H}_3, \beta^{34}\mathbb{H}_3, \dots, \beta^{126}\mathbb{H}_3 \\ \text{Cells}(0, 96-127) &\Leftrightarrow \beta^3\mathbb{H}_3, \beta^{67}\mathbb{H}_3, \beta^{35}\mathbb{H}_3, \dots, \beta^{127}\mathbb{H}_3 \\ \text{Cells}(0, 128-159) &\Leftrightarrow \mathbb{H}_3, \beta^{64}\mathbb{H}_3, \beta^{32}\mathbb{H}_3, \dots, \beta^{124}\mathbb{H}_3 \end{aligned}$$

(b) Local code 2: Cells(0, 128-223)

$$\begin{aligned} f_2(X) &\Leftrightarrow \text{Cells}(0, 64-95) \text{ and Cells}(0, 128-159) \text{ of blob 0} \\ \text{Cells}(0, 128-159) &\Leftrightarrow \mathbb{H}_3, \beta^{64}\mathbb{H}_3, \beta^{32}\mathbb{H}_3, \dots, \beta^{124}\mathbb{H}_3 \\ \text{Cells}(0, 160-191) &\Leftrightarrow \beta\mathbb{H}_3, \beta^{65}\mathbb{H}_3, \beta^{33}\mathbb{H}_3, \dots, \beta^{125}\mathbb{H}_3 \\ \text{Cells}(0, 192-223) &\Leftrightarrow \beta^2\mathbb{H}_3, \beta^{66}\mathbb{H}_3, \beta^{34}\mathbb{H}_3, \dots, \beta^{126}\mathbb{H}_3 \end{aligned}$$

(c) Local code 3: Cells(0, 192-255) and Cells(0, 0-31)

$$\begin{aligned} f_3(X) &\Leftrightarrow \text{Cells}(0, 64-95) \text{ and Cells}(0, 128-159) \text{ of blob 0} \\ \text{Cells}(0, 192-223) &\Leftrightarrow \beta^2\mathbb{H}_3, \beta^{66}\mathbb{H}_3, \beta^{34}\mathbb{H}_3, \dots, \beta^{126}\mathbb{H}_3 \\ \text{Cells}(0, 224-255) &\Leftrightarrow \beta^3\mathbb{H}_3, \beta^{67}\mathbb{H}_3, \beta^{35}\mathbb{H}_3, \dots, \beta^{127}\mathbb{H}_3 \\ \text{Cells}(0, 0-31) &\Leftrightarrow \mathbb{H}_3, \beta^{64}\mathbb{H}_3, \beta^{32}\mathbb{H}_3, \dots, \beta^{124}\mathbb{H}_3 \end{aligned}$$

Observe that local code 3 wraps around cyclically in the sense that it binds together the data in cells in the right end to the ones on the starting left. This is indicated in Fig.5. We remark that the encoding is systematic i.e., data in the blob is available as such in the extended blob.

The exact algorithm using FFT is presented next. Let us use $\mathbf{u} \in \mathbb{F}_q^{64 \times 128}$ to denote blob 0. We use $\mathbf{u}[\text{start-end}]$ to denote the data at the cells Cell(0, start) to Cell(0, end). Let

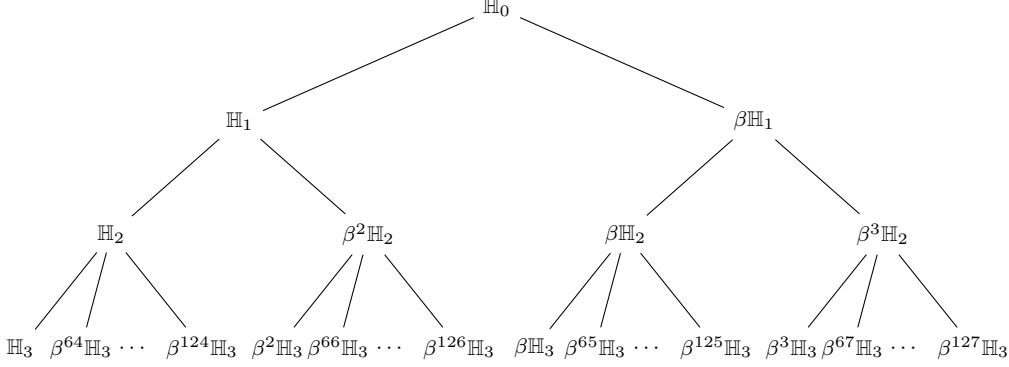


Figure 6: Tree of subgroups for BC code. Here, β is a generator of $\mathbb{H}_0$.

us define four $(D \times \omega) = (64 \times 32)$ blob data arrays in their vectorized form, i.e., as 2048 long vectors:

$$\mathbf{u}_0 = \mathbf{u}[0-31], \quad \mathbf{u}_2 = \mathbf{u}[64-95], \quad \mathbf{u}_4 = \mathbf{u}[128-159], \quad \mathbf{u}_6 = \mathbf{u}[192-223].$$

The extended blob is denoted by $\hat{\mathbf{x}} \in \mathbb{F}_q^{64 \times 256}$ and we have

$$\hat{\mathbf{x}}_i = \hat{\mathbf{x}}[32i - (32i+31)], \quad i = 0, 1, \dots, 7.$$

again in vectorized form. We use $\|$ to concatenate two vectors one after another to form a longer vector. The algorithm is given below.

For $i = 0, 1, 2, 3$, execute:

1. If i is even, set $\mathbb{G} = \beta \mathbb{H}_2$, $j = 2i, \ell = (2i + 2) \bmod 8$
2. If i is odd, set $\mathbb{G} = \beta^3 \mathbb{H}_2$, $j = (2i + 2) \bmod 8, \ell = 2i$
3. Set $((\hat{\mathbf{x}}_j \| \hat{\mathbf{x}}_\ell)(\alpha), \alpha \in \mathbb{H}_1) = \mathbf{u}_j \| \mathbf{u}_\ell$
4. Compute $\mathbf{x}_j \| \mathbf{x}_\ell = \text{IFFT}_{\mathbb{H}_1}(\hat{\mathbf{x}}_j \| \hat{\mathbf{x}}_\ell)$
5. Compute $\hat{\mathbf{x}}_{2i+1} = \text{FFT}_{\beta \mathbb{H}_1}(\mathbf{x}_j \| \mathbf{x}_\ell, \mathbb{G})$. Entries of $\hat{\mathbf{x}}_{2i+1}$ occupy cells respecting the coset assignment to each cell.

The above BC encoder requires four IFFT computations over $\mathbb{H}_1$ and four restricted FFT computations over $\beta \mathbb{H}_1$ restricted to a coset of $\mathbb{H}_2$. It may be noted again that $|\mathbb{H}_1| = k_0 D$ and $|\mathbb{H}_2| = k_0 D/2$.

4.3 A Reconstruction Algorithm

Being 1D RS code, each of four local codes can recover from at most ρD missing symbols. Going beyond, the BC code can recover from any $2\rho D$ missing symbols. We are interested in correcting erasure patterns in which whole columns are missing. In the following, we provide an efficient algorithm below to reconstruct any 2ρ columns (i.e., $2\rho D$ symbols).

The algorithm invokes sequential recovery of missing symbols first relying on each local code, and then on pairs of adjacent local codes. In either case, it invokes 1D RS reconstruction algorithm (See Sec. 2.3). It is given below in steps.

1. Suppose columns indexed by $J, |J| \leq \rho$ are missing in Local code i , $i = 0, 1, 2, 3$. Consider a $([n'_0 = 2(\omega + \rho), k'_0 = 2\omega], D)$ stacked 1D RS code $\mathcal{C}'$ with $\mathbb{H}_0$ as the set of evaluation points. Observe that $\mathcal{C}'$ can recover from any 2ρ column erasures. Assume that the codeword of Local code i comes from $\mathcal{C}'$ with missing columns at J' where $J' \supset J$ and $|J'| = |J| + \rho \leq 2\rho$. Invoke the 1D RS reconstruction algorithm for $\mathcal{C}'$ to recover all missing columns.

2. Suppose that there are still missing columns that could not be handled by any of the local codes. In this case, every local codeword subjected to erasures has $> \rho$ missing columns. It must be that there can be at most two adjacent local codewords of such kind since the total number of erased columns was $\leq 2\rho$ to begin with.

Let Local code i and Local code i' where $i' = (i + 1) \bmod 4$ be those codes. Let $\mathbf{y} = (\mathbf{y}_0, \mathbf{y}_1, \dots, \mathbf{y}_{3\omega-1})$ and $\mathbf{y}' = (\mathbf{y}'_0, \mathbf{y}'_1, \dots, \mathbf{y}'_{3\omega-1})$ be respective codewords subjected to erasures, i.e., some entries in $\mathbf{y}$ or $\mathbf{y}'$ may be missing. The following must be true: (a) None among the first ω columns in $\mathbf{y}$ and the last ω columns in $\mathbf{y}'$ are missing since codewords of Local codes $(i - 1) \bmod 4$ and $(i' + 1) \bmod 4$ are already recovered in Step 1. (b) $\mathbf{y}_j = \mathbf{y}'_{j-2\omega}$ for every $j = 2\omega, 2\omega + 1, \dots, 3\omega - 1$ since Local codes i and i' are adjacent.

- (a) Construct $\mathbf{z} = (\mathbf{y}_0 - \mathbf{y}'_{2\omega}, \mathbf{y}_1 - \mathbf{y}'_{2\omega+1}, \dots, \mathbf{y}_{\omega-1} - \mathbf{y}'_{3\omega-1}, \underbrace{\mathbf{0}, \dots, \mathbf{0}}_{\omega \text{ times}})$
- (b) Interpolate $\mathbf{z}$ to obtain $g(X) = f_i(X) - f_{i'}(X)$ over $\mathbb{H}_2$.
- (c) If i is even, set $\mathbb{G}' = \beta^2\mathbb{H}_2, \mathbb{G}'' = \mathbb{H}_2$, or else set $\mathbb{G}' = \mathbb{H}_2, \mathbb{G}'' = \beta^2\mathbb{H}_2$. Evaluate $g(X)$ over $\mathbb{G}''$.
- (d) Compute $f_i(x) = f_{i'}(x) + g(x), x \in \mathbb{G}''$ at those x where $f_{i'}(x)$ is available from $\mathbf{y}'$.
- (d) Evaluations of $f_i(X)$ over $\mathbb{G}''$ obtained in Step 2(d) add to the currently known evaluations of $f_i(X)$ over $\mathbb{H}_1 \cup \mathbb{G}'$ from $\mathbf{y}$. In this way, we obtain the vector $\mathbf{w} = (f_i(x), x \in \mathbb{H}_0)$ subjected to at most 2ρ erased columns. Invoke 1D RS reconstruction for $\mathcal{C}'$ on $\mathbf{w}$ to recover back $f_i(X)$ and hence all missing columns of $\mathbf{y}$.
- (e) By now, $\mathbf{y}'(\mathbb{H}_1) = (\mathbf{y}'_0, \mathbf{y}'_1, \dots, \mathbf{y}'_{\omega-1}, \mathbf{y}'_{2\omega}, \dots, \mathbf{y}'_{3\omega-1})$ are available. Interpolate $\mathbf{y}'(\mathbb{H}_1)$ over $\mathbb{H}_1$ to recover back $f_{i'}(x)$, and hence all remaining missing columns in $\mathbf{y}'$.

4.4 KZG Commitments for Stacked BC Code

In every column of the stacked BC code, we have evaluations over a subgroup or a coset of a subgroup. Therefore, the KZG multiproof used in 1D RS code is applicable here as well. It is sufficient to have KZG commitments for each local RS code with a blocklength much smaller than the blocklength of the entire BC code. In this way, we reduce the complexity of KZG related computations.

5 Performance Analysis

Consider a BC code with parameters μ, ω and ρ . We fix the overlap factor $\lambda = 2$ throughout the discussion. Recall that μ represents the number of local codes, ρD represents the number of redundant symbols in every local code, and ωD represents the number of symbols at which two adjacent local codes intersect. The rate R , stopping rate S and local code size L of a BC code are given by:

$$\begin{aligned} L &= (2\omega + \rho)D \\ R &= \frac{\mu\omega D}{\mu(\omega + \rho)D} = \frac{\omega}{\omega + \rho} \\ S &= \frac{2\rho D}{\mu(\omega + \rho)D} = \frac{2\rho}{\mu(\omega + \rho)} = \frac{2(1 - R)}{\mu}. \end{aligned}$$

The tradeoff between S and R for a BC code with different values of μ is shown in Fig. 7. The plots also contain the tradeoff for 1D and 2D RS codes. Thus the BC code achieves a region between 1D and 2D RS codes.

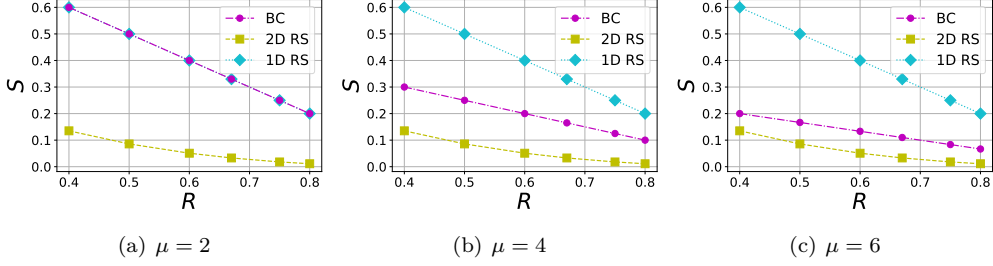


Figure 7: The tradeoff between stopping rate (S) and rate (R) in 1D RS, 2D RS and BC codes with different values of μ .

The absolute maximum number E of missing symbols that can be reconstructed within an extended blob is given by

$$E = nDS \quad (8)$$

where nD is the total number of symbols present in an extended blob and S is the stopping rate. We consider three different blob sizes 1MB, 8MB and 32MB, and each of them maps to a certain number of finite field symbols kD within a blob. The mapping depends on the choice of finite field $\mathbb{F}_q$ because a symbol in $\mathbb{F}_q$ is represented using $\lceil \log_2(q) \rceil$ bits. For example, an 1MB blob is associated with 2^{15} finite field symbols when the finite field is chosen to be the one linked to b1s12-381 with $\lceil \log_2(q) \rceil = 32 \times 2^8 = 32$ bytes. For various file sizes, we plot E versus R in Fig. 8 and Fig. 9. In Fig. 8, a BC code with $\mu = 4$ is compared against 1D and 2D RS codes whereas in Fig. 9, a BC code with $\mu = 6$ is chosen for comparison. It is no surprise that BC code is inferior to 1D RS code because 1D RS code is optimal with respect to E for a given rate. What is interesting is that BC codes with both $\mu = 4$ and $\mu = 6$ perform significantly better than the corresponding 2D RS code for the same file size and rate.

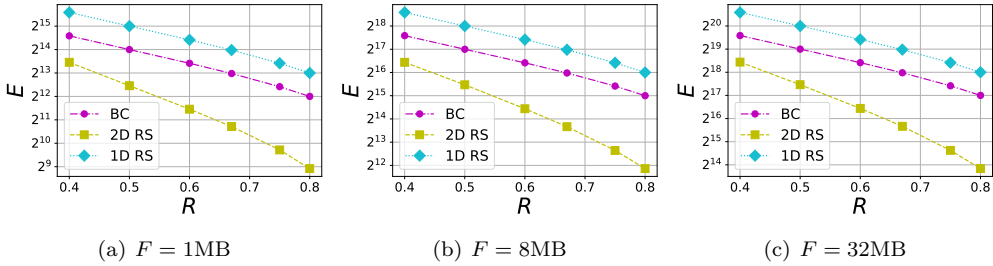


Figure 8: The tradeoff between tolerable number of erasures (E) and rate (R) in 1D RS, 2D RS and BC codes with different values of file sizes and $\mu = 4$.

In Fig. 10, the size of local codes within BC, 1DRS and 2DRS codes are compared. Since an 1D RS code does not contain any local code, the entire code is considered as its local code. It may be observed that 2D RS codes are significantly better than the BC code in absolute size of local code. In summary, a BC code gives a significant advantage in increasing the number of tolerable erasures with regard to the 2D RS code. However, the local code size of a BC code is very large compared to 2D RS code. In any case, the BC code offers a regime of operation different from that of both 1DRS and 2DRS codes, and

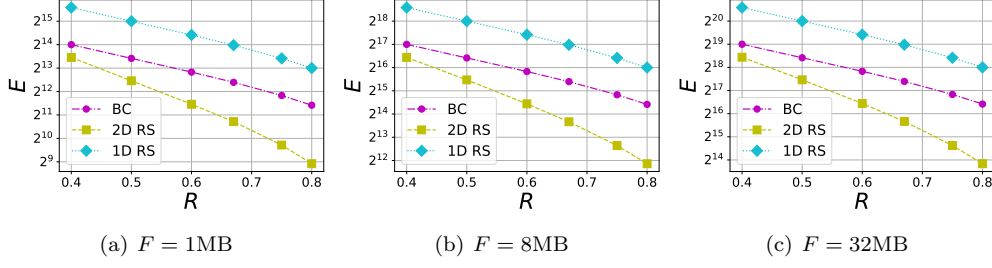


Figure 9: The tradeoff between tolerable number of erasures (E) and rate (R) in 1D RS, 2D RS and BC codes with different values of file sizes and $\mu = 6$.

opens up the possibility of designing codes operating on the intermediate regime between 1D and 2D RS codes.

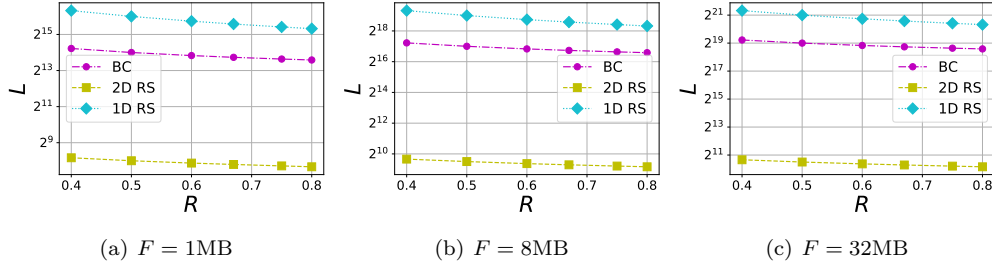


Figure 10: The tradeoff between local code size (L) and rate (R) in 1D RS, 2D RS and BC codes with different values of file sizes and $\mu = 6$. For 1D RS there is no local code, so we take $L = nD$.

Thanks

We thank Yijin Lee for all the support provided during the course of the project.

References

- [ASBK21] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. Fraud and data availability proofs: Detecting invalid blocks in light clients. In *Proc. of the 25th Int. Conf. on Financial Cryptography and Data Security*, volume 12675, pages 279–298, 2021.
- [But18] Vitalik Buterin. Reed-solomon erasure code recovery in $n * \log^2(n)$ time with FFTs, 2018. Accessed: 2025–11–23.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Proc. of the Advances in Cryptology - ASIACRYPT 2010*, volume 6477, pages 177–194, 2010.
- [SVHD25] Birenjith Sasidharan, Emanuele Viterbo, and Son Hoang Dau. Block circulant codes with application to decentralized systems. *IEEE Transactions on Communications*, 73(10):8542–8557, 2025.

-
- [WZ24] Benedikt Wagner and Arantxa Zapico. A documentation of Ethereum’s Peer-DAS. *IACR Cryptol. ePrint Arch.*, page 1362, 2024.